HoloDeck Stream Investigation Summary

Links

Repo: https://github.com/AlysonNumberFIVE/protobuf-forensics

Table of Content

Tooling

The following 2 programming languages can be found being used in the repo with a brief description of what they're used for:

- **Go**: hand-rolled Protocol Buffers decoder (`readVarint`, `readTag`, `parseEvent`) for the event log; verb counting, presence modelling, and duplicate detection. -
- **C**: raw `.mjr` / RTP walker; frame and RTP header parsing, dual-clock drift measurement, sequence-continuity and marker-bit checks.

Introduction

The following is a summary and writeup of the HoloDeck investigation task. It provides a detailed explanation of the investigation, my methodology, and the assumptions that led me to my conclusions.

The issues I found across the project files; the "json" payload and the raw `.mjr` files, are as follows:

- The mislabeled JSON file that turned out to be a Protocol Buffers blob.
- Protobuf wire-type conformance checks.
- Suspicious join events.
- Validating RTP and `.mjr` file integrity.

- Audio/video timestamp falsification.

I unpack each below.

Why the reasoning matters: throughout, I've tried to record not just *what* I checked but *why*; providing the signal that made me look in a given direction.

The Mislabeled JSON File

Directory: protobuf_inspection_go/

See functions:
- main

Among the project files is `c5197a1e-30f4-4dfa-b98f-79a5c88f2fc8.json`. The first step was to attempt to unpack it as JSON. A quick inspection with `cat`, however, showed a lot of unprintable binary data and a JSON file must begin with `{`, `[`, or whitespace, never raw binary. That mismatch between the extension and the contents was the first clue that the file is mislabeled on purpose.

My next guess was that it might be a mislabeled Protocol Buffers blob. This was the natural conclusion given the other files in the project: two `.mjr` files and, tellingly, a `.proto` file. The `event.proto` is effectively the key, telling me the file labeled `.json` is really a serialized `HolodeckStream`. To confirm it, I would manually unpack the blob against the structure the schema defines.

Unpacking the HolodeckStream

See functions:
- readTag
- readVarint
- parseEvent

Because this task leans toward a forensic-style investigation, I went in on the assumption that the file might have problems that make unpacking it with the official protobuf libraries difficult, or worse, that a library would silently paper over the very anomalies I was searching for. Since I only needed to verify that the file is a protobuf blob, I chose to validate it manually, reading every byte myself rather than trusting a decoder to tell me it was fine.

A short summary on manually unrolling a Protobuf blob

Protocol Buffers blobs follow a specific, repeating pattern. If the blob conformed to that pattern, at least for the first object, I could be confident it was genuinely a protobuf blob.

Before unpacking, a brief note on the two things encoded in front of every field.

Field Number and Wire Type Parsing

Before each protobuf message, the very first byte contains the data for a field number and wire type that denotes the specific field of the protobuf message as well as the kind of data of said value. For our specific messages;

The **field number** identifies the field values within `event.proto`:

- Field number 1 - `events` in `HolodeckStream`
- Field number 1 - `timestamp` in `HolodeckEvent`
- Field number 2 - `update` in `HolodeckEvent`
- Field number 3 - `event` in `HolodeckEvent`

The **wire type** tells the parser how to read the bytes that follow:

- **Wire type 0** - a variable-length integer (varint). Each byte contributes 7 bits of value; the 8th bit is a continuation flag. The parser reads bytes one at a time until it hits a byte with the continuation bit cleared. In this file, this is how the raw `timestamp` is read.
- **Wire type 2** - a length, followed by that many bytes of data. Used for `update`, `event`, and the nested `HolodeckEvent` records.

Each field is prefixed by a single **tag** byte encoding that computes the field number with the equation:

```
field_number = tag >> 3
```

And the wire type with the equation:

```
wire_type = tag & 0x07
```

The **wire type** says *how* to read the following bytes and the **field number,** looked up in `event.proto`, says *what* the bytes mean. Together they make the stream self-navigable; every

field's boundaries are discoverable without trusting the payload, which is also what let me audit for unexpected or injected fields.

Parsing the “JSON” blob; A Practical Example

For the blob provided, the first bytes are:

0a 09 08 de a8 c8 8e fb 33 12 00 0a 09…

Stepping through them:

Bytes	Tag	Meaning
`0a`	field 1, wire type 2 (the next number is how many bytes to read)	`HolodeckStream.events` - a length-delimited element follows
`09`	length = 9 (read 9 bytes)	this `HolodeckEvent` is 9 bytes long
`08`	field 1, wire type 0 (read until the continuation bit clears)	`HolodeckEvent.timestamp`: a varint follows
`de a8 c8 8e fb 33`	value	timestamp = 1785394762846 (varint, read via continuation bits)
`12`	field 2, wire type 2 (the next number is how many bytes to read)	`HolodeckEvent.update`: length-delimited
`00`	length = 0 (empty)	empty update
`0a`	field 1, wire type 2	next `HolodeckEvent` begins and the pattern repeats

The blob continues in this pattern, as `events` is a `repeated` field.

Manually assessing the first message gave me the confidence to conclude this was indeed an intact protobuf blob.

Decoding all **1519 events** without error confirmed the stream is well-formed and conforming. The tampering (if any) therefore lives at the *semantic* layer (the values), not the *structural* layer (the framing). I then unpacked the file's contents into a list of custom `HolodeckEvent` structs before continuing.

Suspicious Join Events

See functions:

- frequencyOfActionByKeyValuePair
- contiguousDuplicates
- clusteredDupes

The next step was getting to know the shape of the data and establishing a baseline of what it *should* look like, so anomalies would stand out against it.

Initial exploration surfaced a few strings in the event arrays: `signalling`, `join`, and `leave`. I deduced their meanings as:

- **signalling** - actions inside the Lessonspace canvas: mouse movement, pointing, dragging, microphone activity, etc.
- **join** - a user joining the session.
- **leave** - a user leaving the session.

To gauge the size of the session, I counted occurrences of each label: (terminal output)

map[:198 join:177 leave:3 signalling:1141]

- 198 non-event types (the `update` fields).
- 177 joins.
- 3 leaves.
- 1141 signalling.

177 joins against 3 leaves is the immediate anomaly. Inspecting the payloads for joins, leaves, and signalling, one detail jumped out: in the sample dump, every single join carried the **same `id`**. So my next step was to check *all* the joins for that shared ID.

Inspecting join frequency:

type: join, id: 3740984, frequency: 177

This means every join in the session originates from the exact same user, "Tutor". There are no joins from any other user. The leaves tell the same story:

type: leave, id: 3740984, frequency: 3

This is what makes the imbalance significant. If a single user is joining and leaving, joins and leaves should roughly balance (one join per leave). Instead, one user appears to join 177 times against only 3 leaves which, normally, isn't possible. **You cannot join a room you are already in.** Modelling presence (tracking whether the user is currently in the room), at most ~4 joins can

be legitimate, the session start plus one rejoin per leave. The remaining **~173 joins fired while the user was already present.**

Finally, I wanted to know how *contiguous* these joins are. Inspecting consecutive duplicate payloads:

```
consecutive duplicate payloads: 163
  join       97
  signalling 66
```

The joins aren't just frequent, they're the **exact same payload repeated.** To see how tightly they cluster, I grouped byte-identical payloads across the whole stream and measured the time window each group spans:

```
repeated payloads (count / span / density):
  x39  span=357790 ms  0.1/s     av-talking (state:false)
  x39  span=269845 ms  0.1/s     av-talking (state:true)
  x19  span=5411 ms    3.5/s     mm (mouse move)
  x13  span=14774 ms   0.9/s     join
  ...
  x8   span=6 ms       1333.3/s  join
  x6   span=2 ms       3000.0/s  join
```

Each row groups byte-identical payloads found anywhere in the stream:

- **count** - how many times this exact payload occurs across the session.
- **span** - the elapsed time between first and last occurrence, in milliseconds (the width of the window they fall in).
- **density** - the average occurrences per second (`count ÷ span`), i.e. how tightly packed the copies are.

Density is the key discriminator. A low value means the copies are spread out and are consistent with normal repeated activity, like a user toggling their microphone (the `av-talking` rows, 0.1/s over hundreds of seconds). A very high value means they were emitted in an impossibly tight burst (up to **3000 per second** for the joins). No client, reconnect, or network path produces identical joins that fast. That density is the signature of programmatic injection rather than genuine user actions.

Validating the `.mjr` File Format

The code for this section is in the provided C files and, unlike the previous Go section that focused on debugging the data from the perspective of the session itself, this C section focuses on evaluating the integrity of the .MJR/RTP file structure themselves.

Directory: binary_inspection_c/

Verifying the file structure is intact

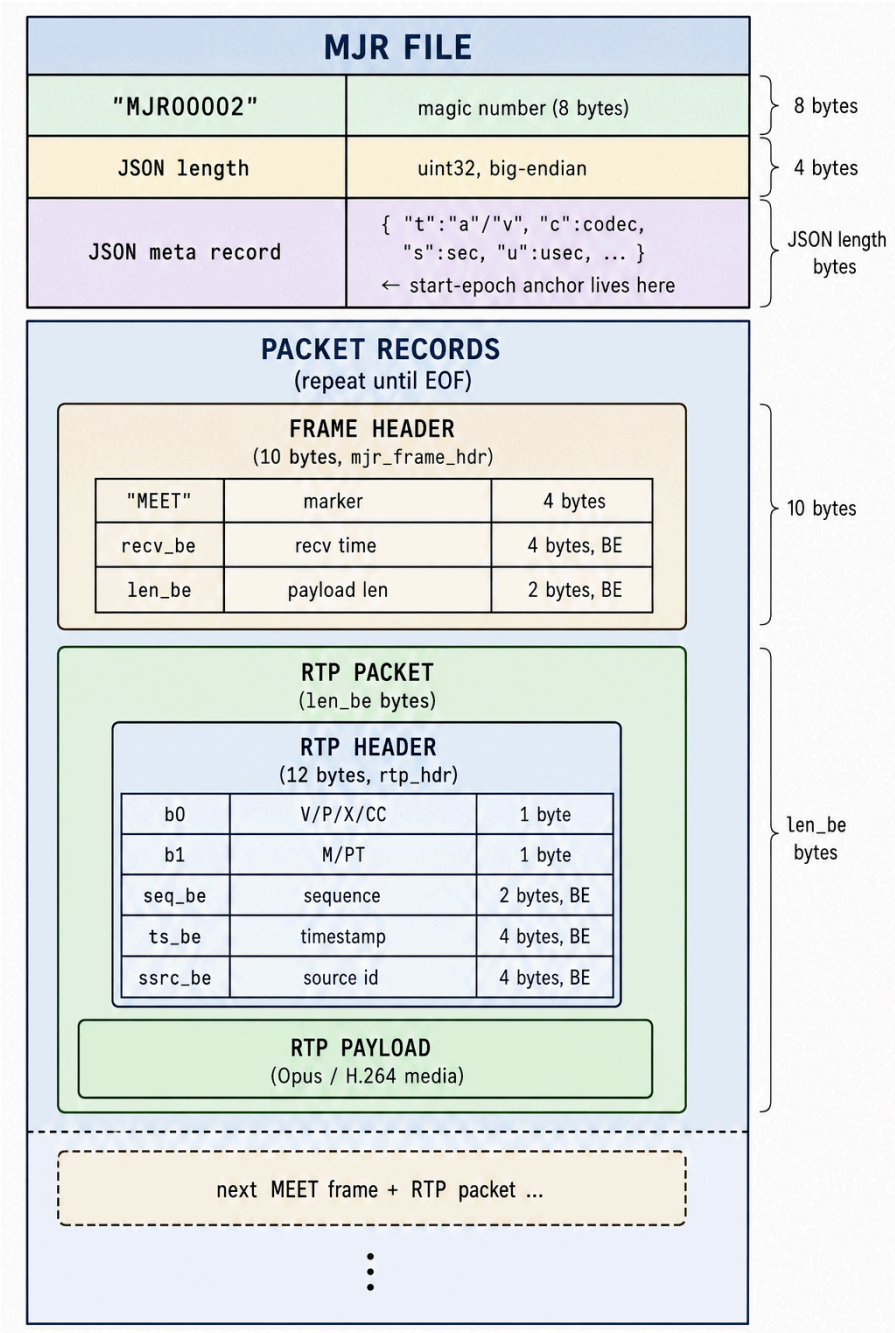
See functions:

- unpack_mjr
- traverse_binary

Because this exercise is about finding issues in a corrupted session, I validated the structure of the `.mjr` files by manually parsing the binary, and I did it by hand deliberately. **A common signature of a corrupted or tampered binary is offsets that don't line up:** headers that don't land where the preceding length fields say they should. A library that abstracts the traversal away would hide exactly that symptom. By walking the file myself, using each frame's own length to predict where the next frame must begin, any structural tampering would derail the traversal and reveal itself.

An `.mjr` file has the following structure:

MJR FILE



The first step was verifying both `.mjr` files are traversable at all. I checked that the magic number is intact; the first 8 bytes match `"MJR00002"`, then, after confirming the JSON length and meta record via struct casting, I walked every packet record. For each frame I verify the frame header begins with `"MEET"`, then advance the traversal pointer by exactly the frame header (10 bytes) plus the `len_be` value from that header. That lands the pointer at the next RTP packet, and I repeat across all frames.

The reasoning: if the file were structurally tampered with, this would break. I'd either reach EOF with leftover bytes, or fail to find `"MEET"` after a frame + `len_be` jump. In this case, a complete traversal succeeds in both the audio and video files. The structure is intact, which tells me the tampering is in the *contents*, not the framing.

Validating the accuracy of the binary's contents

See functions:

- traverse_binary
- ssrc_checker
- inspect_video_stream

Each packet carries header fields whose internal consistency can be checked against known invariants; properties a well-formed recording must satisfy. Where an invariant breaks, the data has been corrupted or deliberately altered. I ran three checks across both streams:

Timestamp consistency. Every packet carries two independent time references: `recv_be` in the frame header (wall-clock arrival time) and `ts_be` in the RTP header (the media clock). In an untampered stream these advance together, so comparing them detects time-desync. *(Detailed below.)*

Source consistency (`ssrc_be`). The synchronisation source identifier labels which sender a packet came from and must stay constant for the lifetime of a stream. A change mid-stream would indicate spliced or mixed-in packets from another source. **This check passed**; the source is consistent throughout both streams.

Frame completeness (marker bit). The RTP marker bit (the top bit of `b1`) flags the final packet of a video frame. By tracking it across packets, a frame boundary reached *without* a preceding marker reveals an incomplete frame, dropped tail packets or a stripped marker. The video stream contained **four incomplete frames**:

** video frame incomplete before ts 17132400 (packet 1796, prev m=0)
** video frame incomplete before ts 20454120 (packet 2326, prev m=0)
** video frame incomplete before ts 26040690 (packet 3232, prev m=0)
** video frame incomplete before ts 27832950 (packet 3538, prev m=0)

Why audio skips this check. The marker bit means different things per media type. In video, `m=1` marks the last packet of a frame; the boundary this test relies on. Audio has no multi-packet frames to close; each packet is a self-contained chunk, and its marker bit instead flags the first packet of a talkspurt (speech resuming after silence). With continuous transmission (and no DTX / silence suppression cutting the stream during quiet passages), there are no talkspurt restarts to signal, so `m` stays 0 on every packet. Applying the video rule here would fire on nearly every packet.

Audio Timestamp Falsification (~50s desync)

See function:

- drift_detection
- print_frame_array

A few timestamp issues were detected in the binaries that appear to be falsified.

1. Media clock vs wall clock, within the audio

The two timestamps in each packet should describe the same passage of time. Measured over the *same* packets (first audio packet to last):

recv_be (wall-clock) span: 372.96 s
ts_be   (media-clock) span: 322.08 s
difference: ~50 s

Both figures describe the identical interval, so they must agree in an honest recording. 372 seconds of packets cannot carry only 322 seconds of media timeline unless the media timestamps were altered.

2. Sequence continuity: ruling out packet loss

The ~50s divergence resolves into **103 forward jumps** in the media clock, distributed across the session. The one innocent explanation for a timestamp jump is packet loss; a lost packet legitimately skips the media timeline forward. But a lost packet *also* skips the RTP sequence number.

So I checked the sequence number at every jump. At **all 103**, the sequence number is perfectly continuous (increments by exactly 1). The media clock gains ~50 seconds across a run of packets where not one is missing. That is impossible for a healthy stream. The timestamps were rewritten while the packet stream itself was left intact; the hallmark of hardware level malfunctioning or tampering.

3. The video stream is clean; the tamper is audio-only

Applying the same walker to the video stream, using the correct 90 kHz video RTP clock rate:

recv_be (wall-clock) span: 372.81 s
ts_be   (media-clock) span: 372.88 s   (@ 90 kHz)
difference: ~65 ms

The video media clock tracks wall-clock time to within 65 milliseconds over the whole session. Since audio and video share the same arrival (`recv_be`) timeline, the clean video stream both confirms the wall clock is trustworthy *and* isolates the manipulation to the audio stream alone.

Open Threads: What I Couldn't Deduce

These are the avenues I investigated but could not resolve.

The `update` payloads are binary and I did not fully decode them. Inspecting their structure, I could confirm a consistent per-record header and length-prefixed keys naming whiteboard objects (`rootcalculator`, `rootdice`, etc.), and I extracted their readable content, which is seemingly coherent session activity (a calculator computation, dice rolls, text formatting, etc.). I did not reverse the full binary grammar, so I can't characterise the underlying format with confidence. What I can state: the `update` stream contains no duplicate payloads, unlike the join events, so the injection I found elsewhere did not reach it.

Example output from inspecting output:

```
len=41    00 00 05 82 d4 9b ce 02 00 00 01 27 13 10 72 6f 6f 74 73 68  |...........'..rootsh|
len=39    00 00 05 82 d4 9b ce 02 00 00 01 27 11 0e 72 6f 6f 74 63 61  |...........'..rootca|
len=33    00 00 05 82 d4 9b ce 02 00 00 01 27 0b 08 72 6f 6f 74 64 69  |...........'..rootdi|
len=48    00 00 05 82 d4 9b ce 02 00 00 01 27 1a 17 72 6f 6f 74 67 72  |...........'..rootgr|
len=41    00 00 05 82 d4 9b ce 02 00 00 01 27 13 10 72 6f 6f 74 73 74  |...........'..rootst|
len=36    00 00 05 82 d4 9b ce 02 00 00 01 27 0e 0b 72 6f 6f 74 6c 65  |...........'..rootle|
len=36    00 00 05 82 d4 9b ce 02 00 00 01 27 0e 0b 72 6f 6f 74 6e 75  |...........'..rootnu|
len=44    00 00 05 82 d4 9b ce 02 00 00 01 27 16 13 72 6f 6f 74 74 69  |...........'..rootti|
len=50    00 00 06 c2 d4 9b ce 02 00 01 02 00 01 28 04 02 69 64 02 01  |............(..id..|
len=33    00 00 06 c2 d4 9b ce 02 00 01 02 00 01 28 07 05 6c 61 74 65  |............(..late|
len=34    00 00 06 c2 d4 9b ce 02 00 01 02 00 01 28 08 06 63 75 72 73  |............(..curs|
len=32    00 00 06 c2 d4 9b ce 02 00 01 02 00 01 28 07 05 65 72 72 6f  |............(..erro|
```

- The consistency of the first few bytes for each payload hints at a consistent header format.
- A few strings can be seen, hinting at rootcalculator, rootdice etc.

```
lyson@192-168-0-100 protobuf-forensics % strings c5197a1e-30f4-4dfa-b98f-79a5c88f2fc8.json | head -n 30
"join",{"id":3740984,"guest":false,"readOnly":false,"allowInvite":false,"meta":{"profilePicture":"","name":"Tutor","color":{"r":240,"g":62,"b":62}
ge":null,"av":{}},"nbf":1785394603,"exp":1785999283,"canLead":true,"iat":1785394723,"traceId":"bd9169a4-a278-4d20-913f-9decffc8a777","socketId":"2
02b-3ea4-41c2-97e3-54161075db47"}]
"join",{"id":3740984,"guest":false,"readOnly":false,"allowInvite":false,"meta":{"profilePicture":"","name":"Tutor","color":{"r":240,"g":62,"b":62}
ge":null,"av":{},"breakoutId":"main"},"nbf":1785394603,"exp":1785999283,"canLead":true,"iat":1785394723,"traceId":"bd9169a4-a278-4d20-913f-9decffc
7","socketId":"23f0a02b-3ea4-41c2-97e3-54161075db47"}]
"join",{"id":3740984,"guest":false,"readOnly":false,"allowInvite":false,"meta":{"profilePicture":"","name":"Tutor","color":{"r":240,"g":62,"b":62}
ge":null,"av":{},"breakoutId":"main"},"nbf":1785394603,"exp":1785999283,"canLead":true,"iat":1785394723,"traceId":"bd9169a4-a278-4d20-913f-9decffc
7","socketId":"23f0a02b-3ea4-41c2-97e3-54161075db47"}]
otshared-video
otcalculator
otdiceD
otgraphing-calculator
otsticky-notes
otletters
otnumbers
ottimer-component
"join",{"id":3740984,"guest":false,"readOnly":false,"allowInvite":false,"meta":{"profilePicture":"","name":"Tutor","color":{"r":240,"g":62,"b":62}
ge":null,"av":{},"breakoutId":"main","leading":true},"nbf":1785394603,"exp":1785999283,"canLead":true,"iat":1785394723,"traceId":"bd9169a4-a278-4d
13f-9decffc8a777","socketId":"23f0a02b-3ea4-41c2-97e3-54161075db47"}]
"join",{"id":3740984,"guest":false,"readOnly":false,"allowInvite":false,"meta":{"profilePicture":"","name":"Tutor","color":{"r":240,"g":62,"b":62}
ge":null,"av":{},"breakoutId":"main","leading":true},"nbf":1785394603,"exp":1785999283,"canLead":true,"iat":1785394723,"traceId":"bd9169a4-a278-4d
13f-9decffc8a777","socketId":"23f0a02b-3ea4-41c2-97e3-54161075db47"}]
"join",{"id":3740984,"guest":false,"readOnly":false,"allowInvite":false,"meta":{"profilePicture":"","name":"Tutor","color":{"r":240,"g":62,"b":62}
ge":null,"av":{},"breakoutId":"main","leading":true},"nbf":1785394603,"exp":1785999283,"canLead":true,"iat":1785394723,"traceId":"bd9169a4-a278-4d
13f-9decffc8a777","socketId":"23f0a02b-3ea4-41c2-97e3-54161075db47"}]
"join",{"id":3740984,"guest":false,"readOnly":false,"allowInvite":false,"meta":{"profilePicture":"","name":"Tutor","color":{"r":240,"g":62,"b":62}
ge":null,"av":{},"breakoutId":"main","leading":true},"nbf":1785394603,"exp":1785999283,"canLead":true,"iat":1785394723,"traceId":"bd9169a4-a278-4d
13f-9decffc8a777","socketId":"23f0a02b-3ea4-41c2-97e3-54161075db47"}]
"join",{"id":3740984,"guest":false,"readOnly":false,"allowInvite":false,"meta":{"profilePicture":"","name":"Tutor","color":{"r":240,"g":62,"b":62}
ge":null,"av":{},"breakoutId":"main","leading":true},"nbf":1785394603,"exp":1785999283,"canLead":true,"iat":1785394723,"traceId":"bd9169a4-a278-4d
13f-9decffc8a777","socketId":"23f0a02b-3ea4-41c2-97e3-54161075db47"}]
"join",{"id":3740984,"guest":false,"readOnly":false,"allowInvite":false,"meta":{"profilePicture":"","name":"Tutor","color":{"r":240,"g":62,"b":62}
```

- A basic run of the strings utility shows us a few isolated strings on individual lines. Ruling out the obvious **event** lines, we can assume each of the individual strings rootsticky-notes, rootgraphing-calculator etc. are evidence of what might be in the **updates** messages.

Cross-format correlation. I was not able to use findings from one format to resolve unknowns in the other (i.e. to take a signal our output from the `.mjr` files and answer an open question in the protobuf blob, or vice versa). The most promising attempt was correlating the audio timestamp jumps against the Holodeck event timeline (the `av-talking` voice-activity segments and comparing them against talkspurts in the .mjr audio file as an example), to test whether the tamper was *content-aware* (targeted at specific spoken moments). This requires anchoring the `.mjr`'s relative clock (`recv_be`, ms since recording start) to the Holodeck's absolute epoch timestamps via the `.mjr` meta record's start-epoch (`s`/`u`) fields. I identified the method but did not complete the correlation, so whether the audio manipulation aligns with specific session events remains an open question.

Prevention

I struggled to create an account on the Lessonspace platform itself (I kept getting authentication issues upon signing up) and as its difficult to tell from these artifacts alone whether a bug is planted as replication of a genuine bug or altering for the purpose of this exercise, the following prevention methods are generic fixes from a security-based forensics perspective. Realistically,

most measures make tampering *detectable* rather than impossible as anyone with write access can edit a file and I think detectability is the right goal.

- **Cryptographic integrity at capture.** Sign or HMAC each recording and event stream at write time, over the raw bytes. Any later modification breaks the signature. This single control would have made every finding here detectable, and is the highest-value recommendation.

- **Cross-stream consistency validation at ingest.** Automate the check from the audio finding: verify each stream's media clock tracks its arrival clock, and that audio and video agree. A ~50s divergence would be flagged automatically. As packets weren't lost (the most common cause for time desync I could think of), it's hard to tell how this would occur naturally and not be as a result of manual tampering of artifacts.

- **Server-authoritative event validation.** The server knows room presence state, so it can reject impossible transitions at record time (e.g. a `join` from a user already present). This closes the injected-join vector. At best flag it in real time.

- **Invariant assertions at ingest.** Assert the properties this investigation relied on: monotonic, continuous RTP sequence numbers; media time and sequence number advancing together; complete frames ending with a marker bit; a constant SSRC. Violations surface anomalies immediately.

Created by Alyson Ngonyama.